

 Państwowa Akademia Nauk Stosowanych w Króćnia		Kierunek Informatyka	
Instrukcja do ćwiczeń laboratoryjnych nr:	3	Nazwa przedmiotu: Programowanie w języku Kotlin	
Temat: Programowanie obiektowe w języku Kotlin		Tryb studiów: stacjonarne	
		Czas trwanie ćw. 2x45 min	
Autor materiałów: dr Marcin Skuba			

1. Treści programowe:

Programowanie obiektowe w języku Kotlin: deklaracje klas, tworzenie nowych obiektów, konstruktory, metody, dziedziczenie, modyfikatory dostępu, interfejsy, klasy abstrakcyjne, obiekty do tworzenia Singletonów, delegacja, polimorfizm,

2. Cel zajęć:

Celem zajęć jest zrozumienie zasad oraz poznanie składni programowania obiektowego w języku Kotlin oraz zaobserwowanie różnic w odniesieniu do innych znanych języków programowania obiektowego.

3. Materiały dydaktyczne

I. Klasy i obiekty

Programowanie obiektowe (OOP) w Kotlinie jest bardzo zwarte. Klasa to „projekt” lub „szablon”, a obiekt to konkretny egzemplarz zbudowany według tego projektu.

Wyobraź sobie, że klasa to projekt samochodu (opisuje, co samochód ma i co robi), a obiekty to konkretne auta stojące na parkingu (czerwony Fiat, niebieskie BMW).

```
class Samochod {
    var predkosc: Int = 0
    var marka: String = "Nieznana"
}

fun main(){
    var samochod = Samochod()
    samochod.predkosc=240
    samochod.marka="Ford"
}
```

II. Konstruktory

• Konstruktor Główny (Primary Constructor)

To najbardziej zwarty sposób. Definiujemy go bezpośrednio w nagłówku klasy. Słowa **val/var** wewnątrz nawiasów sprawiają, że parametry stają się automatycznie właściwościami klasy.

```
// Konstruktor główny z wartościami domyślnymi
class Samochod(
    var predkosc: Int = 0,
    val marka: String = "Nieznana"
)

fun main() {
    // Możemy stworzyć obiekt na kilka sposobów dzięki wartościom domyślnym:
    val s1 = Samochod() // predkosc: 0, marka: Nieznana
    val s2 = Samochod(120, "Ford") // predkosc: 120, marka: Ford
}
```

```
val s3 = Samochod(marka = "BMW") // predkosc: 0, marka: BMW
```

Blok Inicjalizujący (init)

Konstruktor główny nie może zawierać kodu (logiki). Jeśli chcesz np. sprawdzić, czy prędkość nie jest ujemna, używasz bloku init. Jest on wywoływany natychmiast po konstruktorze głównym.

```
class Samochod(var predkosc: Int, val marka: String) {
    init {
        println("Tworzę samochód marki $marka")
        if (predkosc < 0) predkosc = 0 // Logika sprawdzająca
    }
}
```

- **Konstruktory Dodatkowe (Secondary Constructors)**

Czasami chcesz mieć możliwość tworzenia obiektu na różne sposoby (np. z samej marki). Konstruktor dodatkowy musi zawsze wywołać konstruktor główny (używając **this**).

```
class Samochod(var predkosc: Int, val marka: String) {

    // Konstruktor dodatkowy (przyjmuje tylko markę, prędkość ustawia na 0)
    constructor(marka: String) : this(0, marka) {
        println("Wywołano konstruktor dodatkowy dla $marka")
    }
}

fun main() {
    val s1 = Samochod("Toyota") // Wywoła konstruktor dodatkowy, a ten wywoła główny
}
```

Ponieważ konstruktor główny zawsze musi być wywołany jako pierwszy (w linii deklaracji przez **this**) możemy zmodyfikować wartość przekazywanego argumentu w czasie wprowadzania do konstruktora głównego:

```
// Przekazanie do konstruktora zmodyfikowanej wartości marka przed wywołaniem głównego konstruktora
constructor(marka: String) : this(0, marka.uppercase()) {
    println("Wywołano konstruktor dodatkowy dla ${this.marka}")
}
```

Możemy również modyfikować wartości argumentów przez wywołanie naszej dodatkowej metody:

```
class Samochod(var predkosc: Int, val marka: String) {
    // Przekazanie do konstruktora zmodyfikowanej wartosci marka przed wywołaniem głównego konstruktora
    constructor(marka: String) : this(0, zmienWartosc( marka)) {
        println("Wywołano konstruktor dodatkowy dla ${this.marka}")
    }
}

// funkcja musi być poza klasą ponieważ wywołując funkcje z klasy jako argument konstruktora
// obiekt klasy nie istnieje i nie można go wywołać (obiekt istnieje po wywołaniu konstruktora)
fun zmienWartosc(marka: String): String{
    return marka.uppercase()
}
```

```
fun main() {  
    val s1 = Samochod("toyota") // Wywoła konstruktor dodatkowy, a ten wywoła główny  
}
```

• **Konstruktor bez słów val/var (Tylko parametry)**

Jeśli w nawiasach nie dopiszesz val lub var, parametry te nie staną się właściwościami klasy – będą dostępne tylko podczas tworzenia obiektu (w bloku init lub przy przypisywaniu do innych zmiennych).

```
class Samochod(p: Int, m: String) {  
    var predkosc: Int = p  
    val marka: String = m.uppercase() // Możemy zmodyfikować dane przed zapisem  
}
```

Rodzaj	Składnia	Kiedy używać?
Główny (Primary)	class Samochod(val marka: String)	Zawsze, gdy to możliwe. Najbardziej "kotliński" i zwięzły sposób.
Blok init	init { ... }	Gdy potrzebujesz logiki (np. walidacji danych) przy starcie.
Dodatkowy (Secondary)	constructor(...) : this(...)	Gdy potrzebujesz alternatywnych sposobów tworzenia obiektu (różne zestawy danych).

III. Metody

Metody to po prostu funkcje zdefiniowane wewnątrz klasy. Mają one dostęp do właściwości tej klasy.

```
class Kalkulator {  
    // Metoda wykonująca działanie  
    fun dodaj(a: Int, b: Int): Int {  
        return a + b  
    }  
  
    // Metoda wypisująca tekst  
    fun pokazInfo() {  
        println("Jestem prostym kalkulatorem.")  
    }  
}
```

IV. Gettery i settery

W Kotlinie **getter** i **setter** działają inaczej niż w Javie czy C++. Nie musisz ręcznie pisać metod takich jak getAge() czy setAge(), ponieważ Kotlin generuje je automatycznie pod maską dla każdej właściwości (property).

Możesz jednak zdefiniować własne, niestandardowe gettery i settery, aby dodać logikę do odczytu lub zapisu danych.

Domyślnie

Kiedy deklarujesz **var** (zmienną), Kotlin tworzy:

- Pole (field) do przechowywania danych.
- Domyślny getter (do pobierania).
- Domyślny setter (do ustawiania).

Przy **val** (stałej) powstaje tylko pole i getter.

jeśli napiszesz taki kod:

```
class Produkt {
    var cena: Double = 0.0
}
```

Kotlin pod spodem automatycznie wygeneruje coś, co odpowiada takiemu zapisowi:

```
class Produkt {
    var cena: Double = 0.0
    get() = field // Standardowy getter
    set(value) { field = value } // Standardowy setter
}
```

Niestandardowy Getter i Setter

Wyobraźmy sobie klasę *Uzytkownik*. Chcemy, aby imię zawsze było zapisywane z dużej litery, a wiek nie mógł być ujemny.

```
class Uzytkownik(imie: String, wiek: Int) {

    // Niestandardowy SETTER
    var imie: String = imie
    set(value) {
        // 'field' to specjalne słowo kluczowe (backing field)
        // reprezentuje rzeczywistą komórkę pamięci
        field = value.replaceFirstChar { it.uppercase() }
    }

    // Niestandardowy SETTER z logiką
    var wiek: Int = wiek
    set(value) {
        if (value >= 0) {
            field = value
        } else {
            println("Błąd: Wiek nie może być ujemny!")
        }
    }

    // Niestandardowy GETTER (wyliczany)
    val czyPelnoletni: Boolean
    get() = this.wiek >= 18
}

fun main() {
    val user = Uzytkownik("adam", 20)

    // Używamy składni kropki, ale pod spodem wywoływane są nasze metody
    user.imie = "marek" // Zadziała nasz setter i zamieni na "Marek"
    user.wiek = -5 // Zadziała nasz setter i zablokuje zmianę

    println("Imię: ${user.imie}") // Marek
    println("Wiek: ${user.wiek}") // 20 (zmiana na -5 została odrzucona)
    println("Pełnoletni: ${user.czyPelnoletni}") // true
}
```

field (Backing Field)

Wewnątrz gettera lub settera nie możesz użyć nazwy właściwości (np. `imie = value`), bo spowodowałoby to nieskończoną rekurencję (setter wywoływałby sam siebie). Dlatego używamy słowa kluczowego **field**, które odnosi się bezpośrednio do miejsca w pamięci.

Właściwości wyliczane (Computed Properties)

Jeśli właściwość nie przechowuje danych, a jedynie je oblicza na podstawie innych (jak czyPełnoletni), nie potrzebuje pola w pamięci. Wtedy definiujemy tylko `get()`.

V. Modyfikatory dostępu

Modyfikatory dostępu w Kotlinie służą do określania widoczności klas, obiektów, interfejsów, konstruktorów, funkcji oraz właściwości (zmiennych). Pomagają one realizować zasadę **enkapsulacji**, czyli ukrywania wewnętrznej logiki przed światem zewnętrznym.

W Kotlinie mamy **cztery** modyfikatory dostępu. Ważna różnica względem Javy: domyślnym modyfikatorem jest `public`, a nie `package-private`.

Modyfikator	Widoczność
<code>public</code>	(Domyślny) Widoczny wszędzie.
<code>private</code>	Widoczny tylko wewnątrz pliku lub klasy, w której został zadeklarowany.
<code>protected</code>	Widoczny wewnątrz klasy oraz w jej podklasach (klasach dziedziczących).
<code>internal</code>	Widoczny w obrębie tego samego modułu (np. w tym samym projekcie/bibliotece).

W kotlinie **nie** istnieją zmienne **pakietowa** (domyślna w Javie). Jeśli nie zapiszemy żadnego modyfikatora wówczas zmienne będzie **publiczna**.

Poniższy przykład ilustruje, jak różne poziomy dostępu chronią dane wewnątrz klasy i poza nią.

```
// Klasa bazowa
open class Konto(val wlasciciel: String) {

    // PUBLIC: Każdy może sprawdzić walutę
    public val waluta: String = "PLN"

    // PRIVATE: Tylko ta klasa widzi PIN. Nawet klasy dziedziczące go nie zobaczą.
    private var pin: Int = 1234

    // PROTECTED: Widoczne tutaj i w klasach, które dziedziczą po 'Konto'
    protected var tajnyKodAutoryzacji: String = "KOD-99"

    // INTERNAL: Widoczne dla każdej klasy w tym samym module (projekcie)
    internal var numerOddzialu: Int = 101

    fun sprawdzPin(wpisanyPin: Int) {
        if (wpisanyPin == pin) println("PIN poprawny")
    }
}

// Klasa dziedzicząca
class KontoPremium(wlasciciel: String) : Konto(wlasciciel) {
    fun pokazDane() {
        println("Waluta: $waluta")           // OK (public)
        println("Kod: $tajnyKodAutoryzacji") // OK (protected)
```

```

        println("Oddział: $numerOddzialu") // OK (internal)
        // println(pin)                    // BŁĄD! (private w klasie Konto)
    }
}

fun main() {
    val konto = Konto("Jan Kowalski")

    println(konto.waluta)           // OK (public)
    println(konto.numerOddzialu)   // OK (internal - jesteśmy w tym samym projekcie)

    // konto.tajnyKodAutoryzacji // BŁĄD! (protected - main nie dziedziczy po Konto)
    // konto.pin                 // BŁĄD! (private)
}

```

Dokładny opis działania

- public

Jeśli nie wpiszesz nic, Twoja zmienna lub funkcja jest publiczna. Oznacza to, że każdy programista, który zaimportuje Twoją klasę, może z niej korzystać.

- private

To najsilniejsza ochrona.

- Jeśli wstawimy private przy zmiennej w klasie – nikt poza tą klasą jej nie dotknie.
- Jeśli wstawimy private przy klasie w pliku – tylko inne funkcje w tym samym pliku .kt będą ją widzieć.

- protected

Działa tak jak private, ale "robi wyjątek" dla dzieci (klas dziedziczących). Jest to kluczowe w programowaniu obiektowym, gdy chcemy, aby podklasa mogła zmieniać stan rodzica, ale reszta świata – nie.

- internal

To unikalny modyfikator Kotlin. Jest bardzo przydatny przy tworzeniu bibliotek. Możesz stworzyć funkcję, która jest potrzebna w całym Twoim projekcie (module), ale nie chcesz, aby użytkownik, który pobierze Twoją bibliotekę, mógł ją wywołać.

Modyfikatory dla "settera"

Kotlin pozwala na bardzo użyteczny trik: możesz mieć publiczną zmienną do odczytu, ale prywatną do zapisu:

```

class Licznik {
    // Odczyt jest publiczny, ale zmiana (set) tylko wewnątrz klasy
    var wynik: Int = 0
        private set

    fun zwieksz() {
        wynik++
    }
}

```

VI. Dziedziczenie

Dziedziczenie to jeden z filarów programowania obiektowego. Pozwala ono stworzyć nową klasę na bazie już istniejącej, przejmując jej cechy (zmienne) i zachowania (metody). Możemy dziedziczyć tylko z jednej klasy tak jak w Javie.

W Kotlinie podejście do dziedziczenia jest bardzo restrykcyjne: **wszystkie klasy są domyślnie "zamknięte" (final)**. Oznacza to, że nie można po nich dziedziczyć, dopóki wyraźnie na to nie pozwolisz.

Słowo kluczowe open

Aby klasa mogła być "rodzicem", musi posiadać modyfikator open. To samo dotyczy metod i właściwości – jeśli chcesz, aby dziecko mogło je zmienić (nadpisać), one również muszą być open.

Przykład praktyczny: System pojazdów

Stwórzmy ogólną klasę Pojazd oraz klasę pochodną Motocykl:

```
// Klasa bazowa (Rodzic)
open class Pojazd(val marka: String) {

    var predkosc: Int = 0

    // Metoda, której NIE można nadpisać
    fun trab() {
        println("Beep beep!")
    }

    // Metoda, którą MOŻNA nadpisać w klasie pochodnej
    open fun poruszajSie() {
        println("Pojazd jedzie przed siebie.")
    }
}

// Klasa pochodna (Dziecko)
// Używamy dwukropka ':' aby wskazać rodzica
class Motocykl(marka: String, val typ: String) : Pojazd(marka) {

    // Nadpisywanie metody rodzica przy użyciu 'override'
    override fun poruszajSie() {
        println("Motocykl marki $marka mknie między samochodami!")
    }

    fun jazdaNaJednymKole() {
        println("Uaaa! Jazda na tylnym kole!")
    }
}

fun main() {
    val mojMotor = Motocykl("Yamaha", "Sportowy")

    mojMotor.trab()           // Odziedziczone po Pojazd
    mojMotor.poruszajSie()    // Własna, nadpisana wersja metody
    mojMotor.jazdaNaJednymKole() // Unikalna metoda Motocykla
}
```

Najważniejsze zasady dziedziczenia

- **Konstruktor Rodzica**

Klasa pochodna musi zawsze wywołać konstruktor klasy bazowej. W powyższym przykładzie robimy to za pomocą : Pojazd(marka). Przekazujemy parametr marka z konstruktora dziecka prosto do rodzica.

- **Słowo kluczowe override**

W Kotlinie nadpisywanie nie jest domyślne. Musisz użyć słowa override, aby kompilator wiedział, że świadomie zmieniasz działanie metody odziedziczonej po rodzicu.

- **Dostęp do metod rodzica (super)**

Jeśli w nadpisanej metodzie chcesz zachować starą logikę i tylko coś do niej dodać, używasz słowa super.

```
override fun poruszajSie() {
    super.poruszajSie() // Wywołuje "Pojazd jedzie przed siebie"
    println("...i robi to bardzo szybko!")
}
```

VII. Polimorfizm przez dziedziczenie (Wielopostaciowość)

Dzięki dziedziczeniu możesz traktować różne obiekty (np. Samochod, Motocykl, Rower) jako ogólny Pojazd.

Klasa bazowa:

```
open class Pojazd {
    open fun jedz() {
        println("Pojazd jedzie")
    }
}
```

Klasy pochodne:

```
class Samochod : Pojazd() {
    override fun jedz() {
        println("Samochód jedzie")
    }
}

class Rower : Pojazd() {
    override fun jedz() {
        println("Rower jedzie")
    }
}
```

Polimorfizm w praktyce:

```
fun start(pojazd: Pojazd) {
    pojazd.jedz()
}
```

Wywołanie:

```
start(Samochod()) // Samochód jedzie
start(Rower())   // Rower jedzie
```

Przykład z listą:

```
val garaz: List<Pojazd> = listOf(Motocykl("Honda", "Cross"), Pojazd("Nieznany"))

for (p in garaz) {
    p.poruszajSie() // Każdy pojazd zachowa się zgodnie ze swoją definicją
}
```

Podsumowanie

Element	Opis
open class	Pozwala na dziedziczenie po tej klasie.
:	Operator wskazujący klasę bazową.
open fun	Pozwala na nadpisanie tej metody w dziecku.

Element	Opis
override	Oznacza, że nadpisujemy metodę rodzica.
super	Odwołanie do kodu klasy bazowej.

VIII. Companion object (Java – static)

W kotlinie nie występuje specyfikator static znany z języka Java do tworzenia składowych klasowych – dostępnych z poziomu klasy (np. Math.sqrt(9)).

Najczęściej używany odpowiednik static w kotlinie to **companion object**:

```
class MyClass {  
  
    companion object {  
        const val MAX = 100  
  
        fun show() {  
            println("Hello")  
        }  
    }  
}
```

Użycie:

```
MyClass.show()
```

Jeśli w kotlinie napiszemy poniższy kod poza klasą będzie on traktowany jak statyczny dla java:

```
const val MAX = 100  
  
fun show() {  
    println("Hello")  
}
```

@JvmStatic (dla interoperacyjności z Javą)

Jeśli potrzebujesz prawdziwego static dla Javy:

```
class MyClass {  
  
    companion object {  
        const val MAX = 100  
        @JvmStatic  
        fun show() {  
            println("Hello")  
        }  
    }  
}
```

IX. Klasy przechowujące dane – data

Słowo kluczowe data mówi kompilatorowi, że tak klasa będzie służyła głównie do przechowywania danych.

```
data class Samochod(var predkosc: Int, val marka: String)
```

Gdy użyjemy słowa `data` Kotlin automatycznie generuje kilka bardzo ważnych metod.:

1. `equals()`
2. `hashCode()`
3. `toString()`
4. `copy()`
5. `componentN()` (do destrukuryzacji)

Zwykła klasa tego **nie ma** (dziedziczy domyślne wersje z `Any`).

Przykład użycia dodanych funkcji:

- **equals**:

```
val s1 = Samochod(200, "BMW")
val s2 = Samochod(200, "BMW")

println(s1 == s2)
```

Jeśli klasa w deklaracji nie ma słowa `data` - wynik będzie `false`. Dzieje się tak, iż porównywane są referencje (adres w pamięci)

Jeśli klasa w deklaracji ma słowa `data` - wynik będzie `true`. Dzieje się tak ponieważ porównywana będzie zawartość pól.

- **toString**:

zwykła klasa - wynik: `Samochod@6d03e736`

data class - wynik: `Samochod(predkosc=200, marka=BMW)`

- **copy** - funkcja dostępna tylko w `data class`

```
val s1 = Samochod(200, "BMW")
val s2 = s1.copy(predkosc = 250)
```

- **destrukuryzacja** – działa tylko dla `data class`

```
val (predkosc, marka) = s1
```

data class – nie używamy gdy klasa:

- ma dużo logiki,
- reprezentuj zachowanie a nie dane,
- chcemy kontrolować np. `equals()` ręcznie

X. Interfejsy

Interfejs to **kontrakt** — określa *jakie funkcje klasa musi posiadać*, ale nie definiuje jej stanu.

Służy do opisywania **zachowania**, które mogą implementować różne klasy.

Jedna klasa może implementować **wiele interfejsów**.

Interfejs definiuje *co obiekt potrafi robić*. Jakie metody musi mieć klasa.

```
interface Pojazd {
    fun jedz()
}
```

Klasa implementująca:

```
class Samochod : Pojazd {
    override fun jedz() {
        println("Jadę")
    }
}
```

```
}
```

```
}
```

W Kotlinie interfejs może mieć:

- metody abstrakcyjne
- metody z implementacją (domyślną)
- właściwości (bez stanu, tylko deklaracja)

Przykład z implementacją:

```
interface Pojazd {  
    fun jedz()  
  
    fun zatrzymaj() {  
        println("Zatrzymano pojazd")  
    }  
}
```

To działa bez default jak w Javie.

Wiele interfejsów

Kotlin pozwala implementować wiele interfejsów:

```
class Amfibia : Pojazd, Plywajacy {  
    override fun jedz() {}  
    override fun plywaj() {}  
}
```

XI. Klasy abstrakcyjne

Klasa abstrakcyjna to **niepełna klasa bazowa**, której nie można tworzyć jako obiektu.

Może zawierać zarówno metody abstrakcyjne (bez implementacji), jak i gotową logikę oraz stan (pola).

Klasa może dziedziczyć tylko po **jednej klasie abstrakcyjnej**.

Klasa abstrakcyjna definiuje *wspólną bazę i częściową implementację* dla klas potomnych.

Może zawierać:

- metody abstrakcyjne
- metody z implementacją
- stan (pola)
- konstruktor

```
abstract class Pojazd(val marka: String) {  
  
    abstract fun jedz()  
  
    fun info() {  
        println("Marka: $marka")  
    }  
}
```

Klasa dziedzicząca:

```
class Samochod(marka: String) : Pojazd(marka) {  
    override fun jedz() {  
        println("Samochód jedzie")  
    }  
}
```

```
}  
}
```

Cecha	interface	abstract class
Konstruktor	Nie	Tak
Przechowuje stan	nie (tylko deklaracja)	Tak
Wiele dziedziczeń	Tak	nie (tylko jedna klasa)
Implementacja metod	Tak	Tak

XII. Polimorfizm przez interfejs

```
interface Zwierze {  
    fun wydajDzwiek()  
}
```

implementacje:

```
class Pies : Zwierze {  
    override fun wydajDzwiek() = println("Hau")  
}  
  
class Kot : Zwierze {  
    override fun wydajDzwiek() = println("Miau")  
}
```

Użycie:

```
fun zrobDzwiek(zwierze: Zwierze) {  
    zwierze.wydajDzwiek()  
}
```

Każda klasa zachowuje się inaczej.

Wywołanie:

```
fun main (){  
    var pies = Pies()  
    var kot = Kot()  
  
    zrobDzwiek(pies)  
    zrobDzwiek(kot)  
}
```

XIII. Singleton - object

Kotlinie singleton tworzy się **najprościej przez słowo kluczowe object**. Nie trzeba pisać prywatnych konstruktorów jak w Java - język robi to za Ciebie.

- Najprostszy singleton:

```
object Logger {  
    fun log(message: String) {  
        println("LOG: $message")  
    }  
}
```

Przykład:

```
object Database {
    init {
        println("Tworzenie połączenia z bazą danych...")
    }

    fun connect() {
        println("Połączono z bazą")
    }
}

fun main() {
    println("Start programu")

    // W tym momencie obiekt jeszcze NIE został użyty
    Database.connect() // tutaj następuje pierwsze użycie
    Database.connect() // nie tworzy się ponownie
}
```

Użycie:

```
Logger.log("Start aplikacji")
```

Logger istnieje tylko w jednej instancji w całej aplikacji.

- Singleton z przechowywaniem stanu:

```
object Counter {
    var count = 0

    fun increment() {
        count++
    }
}
```

```
Counter.increment()
println(Counter.count)
```

Stan jest współdzielony globalnie.

- Singleton jako „statyczne” rzeczy w klasie

Odpowiednik static z Javy - companion object:

```
class MyClass {

    companion object {
        const val MAX = 100

        fun show() {
            println("Hello")
        }
    }
}
```

Użycie:

```
MyClass.show()
```

- **Singleton z inicjalizacją (lazy)**

Domyślnie object jest tworzony przy pierwszym użyciu (lazy).

Możesz też użyć:

```
val instance by lazy {
    SomeClass()
}
```

Przykład:

```
class Database {
    init {
        println("Tworzenie połączenia z bazą danych...")
    }

    fun connect() {
        println("Połączono z bazą")
    }
}

// Deklaracja z lazy
val database by lazy {
    Database()
}

fun main() {
    println("Start programu")

    // W tym momencie obiekt jeszcze NIE istnieje

    database.connect() // tutaj następuje utworzenie obiektu
    database.connect() // tutaj obiekt już nie jest tworzony ponownie
}
```

Różnica: object vs lazy:

- object:

```
object Logger
```

- singleton tworzony automatycznie
- zarządzany przez Kotlin
- prosty i najczęściej używany

- lazy:

```
val Logger by lazy { Logger() }
```

- daje większą kontrolę
- możesz używać w klasach
- możesz przekazać parametry do konstruktora
- możesz kontrolować wątki

Kiedy używać lazy?

- ✓ gdy obiekt jest „ciężki” (np. baza danych)
- ✓ gdy inicjalizacja jest kosztowna

- ✓ gdy może nigdy nie być użyty
- ✓ gdy chcesz kontrolować moment tworzenia

object - gotowy singleton

lazy - singleton tworzony przy pierwszym użyciu

Zadania

Zadanie 1: Zaawansowane konstruktory i logika stanowa

Stwórz klasę `StatekKosmiczny`, która będzie zarządzać parametrami lotu międzygwiezdnego.

- Konstruktor główny (Primary Constructor):
 - Zdefiniuj dwa pola tylko do odczytu (`val`): `nazwa` (`String`) oraz `kodIdentyfikacyjny` (`String`).
 - Dodaj pole zmienne (`var`) o nazwie `aktualnaPredkosc`, którego domyślna wartość wynosi `0.0`.
- Pola dodatkowe i blok `init`:
 - Dodaj pole `statusSystemow` (`Boolean`), które w bloku `init` zostanie ustawione na `true` (co oznacza, że statek przechodzi autodiagnostykę przy starcie).
 - Dodaj pole `rocznikProdukcji` (`Int`), które nie jest inicjowane w konstruktorze głównym.
- Konstruktor wtórny (Secondary Constructor):
 - Stwórz konstruktor dodatkowy, który przyjmuje: `nazwa`, `kodIdentyfikacyjny` oraz `rocznikProdukcji`.
 - Ważne: Ten konstruktor musi wywoływać konstruktor główny (`this(...)`).
- Metody (Logika biznesowa):
 - `zwiększMoc(procent: Double)`: Metoda ma zwiększać `aktualnaPredkosc` o podany procent (np. jeśli prędkość wynosi `100`, a podasz `0.1`, nowa prędkość to `110`).
 - `awaryjneHamowanie()`: Metoda ustawia prędkość natychmiast na `0.0` i zmienia `statusSystemow` na `false`.
 - `wyswietlRaport()`: Metoda wypisuje w konsoli komplet informacji o statku w czytelny sposób (użyj *string templates* z dolarem `$`).

Zadanie 2: Własne gettery i settery

Stwórz klasę `KontoBankowe`.

- Posiada ona prywatne pole `_saldo`.
- Zdefiniuj publiczną właściwość `saldo` z własnym getterem (zwracającym wartość) oraz seterem, który nie pozwoli ustawić `saldo` na wartość ujemną (wyświetl komunikat o błędzie).

Zadanie 3: Modyfikatory dostępu

Stwórz pakiet `biuro`.

- Wewnątrz stwórz klasę `Pracownik` z polami: `imie` (publiczne), `pensja` (**private**) oraz `identyfikator` (**internal**).
- Sprawdź w innym pliku (w tym samym pakiecie i w innym), do których pól masz dostęp.

Zadanie 4: Dziedziczenie i polimorfizm

- Stwórz klasę bazową `Zwierze` (pamiętaj o słowie kluczowym `open`).
- Klasa powinna mieć metodę `wydajDzwiek()`, która wypisuje "Zwierzę wydaje dźwięk".
- Stwórz klasy pochodne `Pies` i `Kot`, które **nadpisują** (override) tę metodę, wypisując odpowiednio "Hau" i "Miau".
- Stwórz listę typu `List<Zwierze>` zawierającą oba obiekty i wywołaj metodę w pętli (**polimorfizm**).

Zadanie 5: Companion Object

Stwórz klasę `User`.

- Klasa powinna mieć prywatny konstruktor.

- Wewnątrz zdefiniuj **companion object**, który będzie pełnił rolę fabryki: metoda `create(name: String)` ma zwracać nową instancję klasy `User`.
- Dodaj w `companion object` stałą `MAX_LOGIN_ATTEMPTS`.

Zadanie 6: Data Class

Stwórz klasę typu **data class** o nazwie `Produkt`.

- Pola: `nazwa`, `cena`, `kategoria`.
- W funkcji `main` stwórz instancję, zrób jej kopię ze zmienioną ceną za pomocą metody `.copy()` i wypisz obiekt do konsoli, aby zobaczyć automatycznie wygenerowaną metodę `toString()`.

Zadanie 7: Klasy abstrakcyjne i Interfejsy

- Stwórz **klasę abstrakcyjną** `Kształt` z metodą abstrakcyjną `obliczPole()`.
- Stwórz **interfejs** `Rysowalny` z metodą `rysuj()`.
- Stwórz klasę `Kolo`, która dziedziczy po `Kształt` i implementuje `Rysowalny`.
- Zaimplementuj wymagane metody.

Zadanie 8: Polimorfizm przez interfejs

- Stwórz interfejs `PojazdElektryczny` z metodą `laduj()`.
- Stwórz dwie klasy: `Tesla` i `Hulajnoga`, które implementują ten interfejs.
- Napisz funkcję `rozpocznijLadowanie(pojazd: PojazdElektryczny)`, która przyjmuje dowolny obiekt implementujący ten interfejs i wywołuje na nim metodę `laduj()`.

Zadanie 9: Singleton w Kotlinie

- Stwórz obiekt typu (Singleton) o nazwie `BazaDanych`.
- Powinien on przechowywać listę stringów (np. logi).
- Dodaj metodę `dodajLog(txt: String)`.
- Wywołaj tę metodę z różnych miejsc w kodzie i udowodnij, że lista w `BazaDanych` jest wciąż ta sama.